Q22: What Separates an LLM from an Agent? Is Claude Code an Agent or a Harness?

DLS 2026 — Module (Paradise)

October 2026

Learning objectives

After this module, a student should be able to (i) place any LLM-based system on the agent spectrum, from a single model call through tool-augmented loops to autonomous goal pursuit, (ii) state the minimal definition of an agent — model, loop, tools, memory, stopping criterion — and say which systems fail it, (iii) distinguish the *harness* (the scaffold managing the loop) from the model it wraps, and (iv) explain what changes technically when the loop is added: non-stationary context, error recovery, compounding context rot, and what this does to evaluation.

1 The spectrum: from model call to goal pursuit

"Agent" is used loosely in practice, but the systems the word covers fall on a fairly clean spectrum of *who controls the control flow*.

1. **Single model call.** A chat completion: prompt in, tokens out. The caller owns everything — context assembly, what happens to the output, whether to try again. A base LLM is, in the classical vocabulary, a mapping from a percept (the prompt) to an action (the next token or completion); it has no persistent state across calls, no goals, and no ability to act on the world by itself [5].
2. **Tool-augmented single call.** The model can emit structured tool requests (function calls), and the runtime executes them and feeds results back [6]. If the result comes back and the process stops there, the model is a *component with side effects*, not yet an agent: control flow is still one pass, owned by the caller.
3. **The agentic loop.** The model's output is fed back as new input, with tool results appended, until a stop condition holds. This is ReAct's structure — interleave reasoning traces and actions, let observations update the next step [10] — and it is the load-bearing pattern of every system marketed as an agent today.
4. **Autonomous goal pursuit.** Long-horizon variants: the system is given a goal rather than a task instance, manages its own memory and subgoals across many episodes, and self-critiques and retries (Reflexion-style verbal reinforcement between episodes) [7]. Surveys of this literature typically decompose such systems into profile, memory, planning, and action modules [8].

Anthropic's own engineering guidance draws the line the same way, splitting "agentic systems" into *workflows* (LLMs and tools orchestrated through predefined code paths) and *agents* (systems where the LLM dynamically directs its own process and tool usage), and recommending the simplest

design that works [1]. The move up the spectrum is a move of *control-flow ownership* from the caller to the system.

2 A minimal definition of an agent

Strip the marketing and a minimal agent is five things:

- **Model:** something that maps the current context to a next action. Today, usually an LLM, but the definition does not require it — the code demo in this module uses a ∼30-line rule-based policy and still produces agent behavior.
- **Loop:** a control structure that feeds results back until a stop condition. Without this you have a function call, not an agent.
- **Tools:** actions with effects on an environment (read a file, run a command, call an API), plus their results as observations. Tools convert text generation into interaction with a world.
- **Memory:** whatever carries information across steps — at minimum the growing transcript, often explicit scratchpads or retrieved context.
- **Stopping criterion:** a decision rule that terminates the loop — a `finish` action, a budget of steps or dollars, a detected error. An agent without a stopping criterion is a `while True` loop.

Each element is a design surface. The model decides *locally* (one step at a time, from the current context); the loop, tools, memory, and stopping criterion jointly determine what the system can accomplish. This split — local decisions in the weights, global behavior in the loop design — is the module's central claim.

3 The harness: the scaffold that owns the loop

Call the software around the model the **harness**: the scaffold that assembles the prompt, dispatches tool calls, appends results, manages context and memory, handles retries and malformed output, enforces permissions, and decides when to stop. The model chooses the next action; the harness owns everything else about the control flow.

Why insist on the distinction? Because the two layers fail differently, are built by different people, and improve independently:

- **Agency is in the loop design, not the weights.** The same frontier model behaves as a chatbot, a workflow step, or a coding agent depending on the loop around it. The progression from single call → tool use → loop is a harness change, not a model change.
- **The harness is where most engineering lives.** SWE-agent's central result is that the *agent–computer interface* — a curated set of edit/search/navigation commands with guardrails and concise feedback, plus context management and malformed-generation retries — improved SWE-bench resolution from about 3.8% (non-interactive retrieval) to 12.47% and beat a raw-shell agent by a wide margin, with the base LM's weights untouched [9].
- **Harnesses are swappable and mix-and-match.** One harness can drive several models; one model can sit under several harnesses. "Which model?" and "which harness?" are separate questions, and conflating them is the most common category error in agent discourse.

4 So is Claude Code an agent or a harness?

Strictly: Claude Code is a *harness* — a terminal-native scaffold around one or more LLMs that manages tools (file edits, shell, MCP servers), permissions, context, and retries, and exposes both an interactive and a headless mode [2]. The *system* (harness + model, running a task) is an agent in the sense of §2–3: the model directs its own process step by step inside the loop. Both statements are true because they describe different layers. The useful phrasing is **agentic harness**: the LLM makes local decisions; the harness owns the control flow. The same analysis applies to OpenAI's Codex CLI and to Hermes Agent (the system writing this module) — each is a scaffold of prompt assembly, tool dispatch, context management, and stopping rules wrapped around a model.

Two caveats keep this honest. First, the boundary moves over time: model-side features (tool-use training, longer context, better instruction-following) absorb work the harness used to do, so today's harness responsibility may be tomorrow's model capability. Second, the word "autonomy" is doing heavy lifting in level 4 of the spectrum; a system that runs for hours still executes one locally-chosen action at a time, and its apparent goal-directedness is the loop plus memory plus a stopping rule, not a persistent internal optimizer [8].

5 What changes when you add the loop

The loop is cheap to describe and expensive to live with. Four technical consequences:

Non-stationary context. In a single call, the context is fixed at request time. In a loop, the context is rewritten every iteration by the agent's own actions: tool outputs, error messages, partial files. The model conditions on a distribution it is itself generating — a feedback loop the base next-token objective never saw in training in this form.

Error recovery as a first-class concern. A chat completion fails visibly; a loop fails *silently and compounding*. A misread filename becomes a wrong edit becomes a failing test becomes a confusing traceback in context. Harnesses therefore invest heavily in retries, validation of tool arguments, and readable error feedback — SWE-agent's guardrails and per-turn feedback exist precisely because agents cannot otherwise recover from their own mistakes [9], and Reflexion shows the recovery can even be pushed back into context as verbal self-critique [7].

Compounding context rot. The transcript grows monotonically, and long-context use is not free: performance on retrieving relevant information degrades when it sits in the middle of long contexts [4]. Each step adds tokens; each added token dilutes attention to earlier constraints; the diluted attention produces worse actions, which produce noisier observations, which further bloat the context. This is why harnesses implement context compaction, summarization, and selective history — Claude Code's own guidance recommends clearing context between tasks for exactly this reason [2].

Compounding error rates. If per-step reliability is p and a task needs k dependent steps, end-to-end success is bounded by roughly p^k: a 95% per-step system over 40 dependent steps succeeds about 13% of the time. Long-horizon competence is therefore hypersensitive to per-step quality — and to harness features (retries, verification, self-correction) that effectively raise p or cut k.

6 Evaluation implications (forward link to Q24)

Everything above breaks the single-call evaluation habit. You cannot score an agent the way you score a model:

- **The unit is the trajectory, not the response.** Two systems with identical final answers can differ in cost, tool discipline, and number of dangerous actions.
- **Harness and model must be evaluated separately.** A leaderboard entry conflates them; the same model under a better harness jumps — SWE-agent's gains came entirely from the interface layer [9]. Comparative claims need a stated harness, tools, prompts, and budgets.
- **Variance and cost are first-class.** Agent benchmarks are small, stochastic, and often overfit; accuracy-only comparisons mislead, and cost–accuracy tradeoffs are real optimization targets [3].

Module Q24 (*How do we judge whether an agent is good?*) takes up benchmark design, leakage, and cost–accuracy measurement in detail; the point established here is *why* that module needs to exist.

7 Takeaway

An LLM is a next-action selector; an agent is what you get when a loop wraps that selector with tools, memory, and a stopping rule. The loop — not the weights — supplies the agency, and the software that manages the loop, the harness, deserves to be named, designed, and evaluated as a first-class artifact. Claude Code is best described as an agentic harness: the model decides, the harness governs. When someone asks "is X an agent?", the productive reframe is: *which layer are you asking about, and who owns the control flow?*

References

[1] Anthropic. Building effective agents. https://www.anthropic.com/engineering/building-effective-agents, 2024. Engineering blog post, December 2024; accessed October 2026.

[2] Anthropic. Claude code: Best practices for agentic coding. https://www.anthropic.com/engineering/claude-code-best-practices, 2025. Engineering blog post, April 18, 2025; accessed October 2026.

[3] Sayash Kapoor, Benedikt Stroebl, Zachary S. Siegel, Nitya Nadgir, and Arvind Narayanan. Ai agents that matter. *arXiv preprint arXiv:2407.01502*, 2024.

[4] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024. arXiv:2307.03172.

[5] Stuart Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. Pearson, 4th edition, 2021.

[6] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can

teach themselves to use tools. In *Advances in Neural Information Processing Systems*, 2023. arXiv:2302.04761.

[7] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, 2023. arXiv:2303.11366.

[8] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, Wayne Xin Zhao, Zhewei Wei, and Ji-Rong Wen. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6):186345, 2024. arXiv:2308.11432.

[9] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems*, 2024. arXiv:2405.15793.

[10] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023. arXiv:2210.03629.